

Next-Generation Reservoir Computing for Dynamical Inference

Rok Cestnik^{1, a)} and Erik A. Martens^{1, 2}

¹⁾Centre for Mathematical Science, Lund University, Märkesbacken 4, 223 62 Lund, Sweden

²⁾IMFUFU, Department of Science and Environment, Roskilde University, Universitetsvej 1, Roskilde, Denmark

(Dated: 12 January 2026)

We present a simple and scalable implementation of next-generation reservoir computing (NGRC) for modeling dynamical systems from time-series data. The method uses a pseudorandom nonlinear projection of time-delay embedded inputs, allowing the feature-space dimension to be chosen independently of the observation size and offering a flexible alternative to polynomial-based NGRC projections. We demonstrate the approach on benchmark tasks, including attractor reconstruction and bifurcation diagram estimation, using partial and noisy measurements. We further show that small amounts of measurement noise during training act as an effective regularizer, improving long-term autonomous stability compared to standard regression alone. Across all tests, the models remain stable over long rollouts and generalize beyond the training data. The framework offers explicit control of system state during prediction, and these properties make NGRC a natural candidate for applications such as surrogate modeling and digital-twin applications.

Reservoir computing (RC)¹ is a machine learning method used to predict the dynamic behavior of complex systems. It relies on a reservoir — a fixed, dynamical system — into which an input signal from a given external system of interest is injected. The output of the reservoir is then passed through a trained *readout layer* in order to predict future states of the external system. Next-generation reservoir computing (NGRC)² is a simplified variant of RC that replaces the nonlinear reservoir with a direct *feature expansion*. Here, *features* refer to measurable properties derived from data, and *feature expansion* means generating additional features by combining existing ones. This simplification makes NGRC more interpretable and easier to control than traditional RC. In this work, we introduce a pseudorandom projection as a simple and scalable way to build these features, and use it to model time-series data. Our approach makes accurate and stable predictions even when observations are noisy or incomplete. Through examples such as chaotic attractor reconstruction, bifurcation mapping, and phase recovery, we illustrate how next-generation reservoir computing can capture essential system dynamics and generalize beyond the training data. Our results highlight their potential as a practical tool for applications ranging from digital twins to models of physical, biological, and financial systems.

I. INTRODUCTION

Reservoir computing (RC) has emerged as a powerful and versatile machine learning framework for processing temporal data, particularly in tasks involving time series prediction and the inference of latent variables in dynamical systems^{3–5}. As Lu *et al.*¹ describe, “*Reservoir computing is a machine-learning approach that has demonstrated success at a variety of tasks, including time series prediction and inferring unmeasured variables of a dynamical system from measured variables*”. In its classical form, RC leverages a high-dimensional, nonlinear dynamical system — referred to as the *reservoir* — which is driven by an input signal. The reservoir’s response is then post-processed with a linear map to obtain the desired output, commonly referred to as *readout*. For a general overview of the theory and state of the art, see also Refs.^{6,7}.

A key advantage of RC lies in its simplicity: the reservoir itself operates as a static component that does not require training, while only the output layer (readout) is optimized, typically through linear regression. This design allows for implementations of the reservoir in both software — often utilizing recurrent neural networks — and physical systems. Such flexibility has led to a wide range of serious physical implementations, including optical systems⁸, electronic delay circuits⁹, and photonic devices¹⁰. And to illustrate just how general the reservoir-computing principle is, researchers have even demonstrated it using simple water tanks where surface ripples act as the reservoir¹¹ — a setup meant more as a proof-of-concept than a practical device. Quantum-inspired implementations of reservoir computing have also been explored in recent work¹². The capacity to treat the reservoir as a black box, without needing a detailed model or adjustable internal parameters, sets RC apart from other machine learning paradigms, such as deep learning.

While these systems have demonstrated impressive ca-

^{a)}Electronic mail: rok.cestnik@math.lth.se

$\mathbf{u}(t) \in \mathbb{R}^N$	System state at time t
$\mathbf{u}_H(t) \in \mathbb{R}^{H \times N}$	Measurement vector of history-embedded system states, containing state vectors at times $t, t - \tau, \dots, t - (H - 1) \cdot \tau$.
$\mathbf{r} \in \mathbb{R}^M$	Reservoir state at time t
$P : \mathbf{u}_H \mapsto \mathbf{r}$	Projection of the measurement vector to the higher-dimensional reservoir space
$p_m \in \mathbb{R}$	Elements of the vector $P(\mathbf{u}_H) = [p_1, p_2, \dots, p_M]$
$\mathbf{P} \in \mathbb{R}^{M \times T}$	Matrix of all projected measurement vectors for the complete dataset
$\mathbf{W}_{\text{out}} \in \mathbb{R}^{M \times N}$	Matrix of the output weights. Prediction model for the linear relation $\mathbf{r} = \mathbf{W}_{\text{out}} \mathbf{u}$
$\mathbf{U} \in \mathbb{R}^{N \times T}$	Matrix of all state vectors for the complete dataset
$\tau \in \mathbb{R}^+$	Time-sampling step
$T \in \mathbb{N}$	Number of time data points
$N \in \mathbb{N}$	System state dimension $\mathbf{u}(t)$
$H \in \mathbb{N}$	Time-embedding dimension
$M \in \mathbb{N}$	Projection dimension, reservoir space dimension, $M \gg 1$
$E \in \mathbb{R}$	Mean square error of one-step predictions, quantifying success of training

TABLE I: List of symbols.

pabilities^{13–17}, classical RC has limitations, particularly in terms of interpretability and control. The internal state of the reservoir is often opaque, and perturbing or probing the system in a controlled manner can be challenging. These limitations have motivated the development of a new paradigm: *next-generation reservoir computing* (NGRC)².

NGRC departs from the traditional architecture by eliminating the *explicit* reservoir altogether. Instead, it constructs a high-dimensional feature space of dimension, M , (corresponding to an *implicit* reservoir) through a deterministic projection of the input signal. This results in a simple, fully transparent mapping from one time step to the next, where the system state is entirely known and controllable. Letting $u(t)$ denote the input and $r(t)$ the projected state, the evolution becomes a direct function $r(t + 1) = F(r(t), u(t))$, enabling precise interventions and measurements. This is particularly advantageous in applications requiring system perturbation and response analysis, such as the computation of phase response curves^{18–22}. See Sec. II and Fig. 1 for a more detailed comparison of the two methods.

When using NGRC a challenge remains to design a feature projection algorithm that is independent of the observation dimension, and the desired number of final features. For example, **suppose the observation is N -dimensional with a time-embedding of H steps. The dimension of the input to the reservoir is then NH . Further, suppose we use a feature expansion built on pairwise combinations, we end up with $(NH)^2$ features. It is desirable that changing the observation dimension, NH , does not require changing the reservoir dimension M , i.e., the method is *scalable*.**

Our work adopts the next-generation reservoir computing framework with an emphasis on scalability and interpretability. In particular, we introduce a pseudo-random nonlinear projection of time-delay embedded input as a simple and dimension-independent alternative to

the polynomial-based projections used previously. This projection maintains flexibility across different system sizes while avoiding the need to manage opaque internal reservoir states. The approach is illustrated on benchmark tasks such as attractor reconstruction, bifurcation diagram estimation, and phase recovery, showing stable long rollouts and robustness to partial or noisy data. We also include an exploratory demonstration of asymptotic phase estimation using NGRC.

The remainder of this paper is structured as follows. In Sec. II, we briefly review classical RC and NGRC approaches. Sec. III explains our methods and introduces the projection method used to lift the input into a high-dimensional space (Sec. IIIB), followed by details on training the linear model (Sec. IIIC) and tracking prediction errors (Sec. IIID). In Sec. IV we present three example applications to demonstrate the method’s capabilities: the reconstruction of a chaotic attractor, the estimation of a bifurcation diagram, and the recovery of asymptotic phases of an oscillatory process. Finally, we close with a discussion of the implications and limitations of our approach in Sec. V.

II. BRIEF REVIEW OF RESERVOIR COMPUTING

We briefly review two reservoir computing approaches which form the conceptual foundation for the methods used throughout this paper: the classical reservoir computing (RC) and the more recent next-generation reservoir computing (NGRC) scheme.

1. Traditional Reservoir Computing (RC)

We first review the classical reservoir computing (RC) framework (Lu *et al.*¹), summarized diagrammatically in Fig. 1a. Let $\mathbf{u}(t)$ denote the input (or measurement)

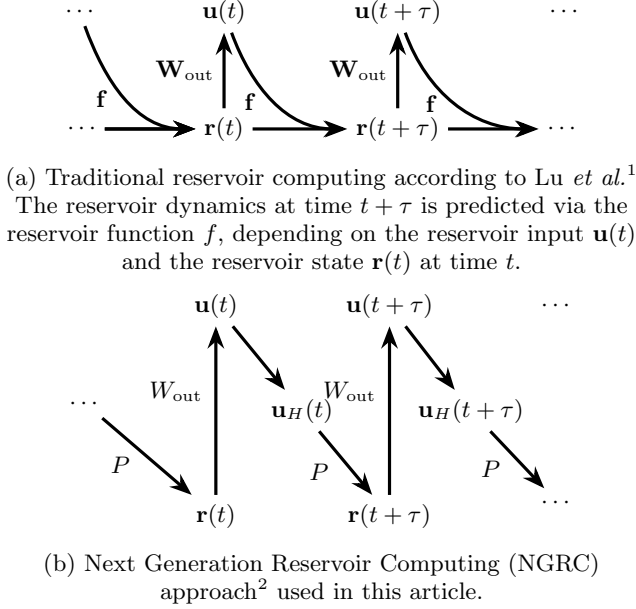


FIG. 1: Comparison of traditional Reservoir Computing (RC)¹ and Next Generation Reservoir Computing (NGRC)² schemes.

vector and $\mathbf{r}(t)$ the internal state of the reservoir. The reservoir evolves according to the dynamical system,

$$\dot{\mathbf{r}}(t) = \mathbf{f}(\mathbf{r}(t), \mathbf{u}(t)), \quad (1)$$

where $\mathbf{f}$ is an appropriately chosen function that is differentiable and typically high-dimensional.

A common choice for the *reservoir function* $\mathbf{f}$ is

$$\mathbf{f} = \gamma(-\mathbf{r} + \tanh(\mathbf{M} \cdot \mathbf{r} + \sigma \mathbf{W}_{\text{in}} \cdot \mathbf{u})). \quad (2)$$

Thus, updating the reservoir $\mathbf{r}$ involves a nonlinear transformation (e.g., $\tanh$) of a linear combination of the reservoir state $\mathbf{r}$ and the input signal $\mathbf{u}$. The input signal, $\mathbf{u}$, is linearly mapped into the reservoir via weights, $\mathbf{W}_{\text{in}}$, and the internal reservoir dynamics, $\mathbf{r}$, is governed by a (possibly random) matrix, $\mathbf{M}$. In practice, the exact form of $\mathbf{M}$ and $\mathbf{W}_{\text{in}}$ does not need to be known, however, they need to remain unchanged throughout the phases of training and prediction.

Reservoirs of this type are commonly referred to as “echo state networks” (ESNs)^{13,23–26}. A key requirement is the *echo state property*^{3,27,28}, which ensures that the reservoir state $\mathbf{r}(t)$ in the asymptotic time limit does not depend on initial conditions but only on the input history. That is, the reservoir must effectively “forget” its initial state in order to respond reliably to the input.

The output of the reservoir, i.e., the estimated system state $\mathbf{u}(t)$, is obtained as a readout from the reservoir state via a linear output layer,

$$\mathbf{u}(t) = \mathbf{W}_{\text{out}} \cdot \mathbf{r}(t),$$

where $\mathbf{W}_{\text{out}}$ is trained, e.g., via linear regression, to learn the mapping $\mathbf{W}_{\text{out}}$ based on time series predictions. Often, the output layer uses a concatenation of $\mathbf{r}(t)$ with its element-wise square, $\mathbf{r} \odot \mathbf{r}$, to break linear symmetries (e.g., sign reversals) and capture nonlinear structure; for further details on RC refer to, e.g., Ref. 1.

In reservoir computing, the equations governing the reservoir dynamics and output are used during both the training and predicting phases. During the *training phase*, the reservoir state $\mathbf{r}(t)$ evolves according to the input signal $\mathbf{u}(t)$, and the output $\mathbf{u}(t) = \mathbf{W}_{\text{out}} \cdot \mathbf{r}(t)$ is used for learning the mapping represented by the weight matrix $\mathbf{W}_{\text{out}}$. In the *prediction phase*, the trained output layer $\mathbf{W}_{\text{out}}$ is applied to the reservoir state to determine the new input signal $\mathbf{u}(t)$, which then in turn continues drive the evolution of the reservoir. (We note that some literature distinguishes variables for the training and prediction phases, but for our purposes we maintain the same notation for $\mathbf{r}(t)$ and $\mathbf{u}(t)$, regardless of whether they represent values for training or prediction phases.) In this context, readout weights are parameters optimized during the training phase, whereas the projection dimension M and other process defining quantities often are referred to as *hyperparameters*, and are set before training and held fixed during prediction.

One of the main advantages of RC over traditional recurrent neural networks is that training is simple: only the output layer $\mathbf{W}_{\text{out}}$ needs to be trained (learned), while the reservoir dynamics itself remains fixed and untrained. For the readout, linear mapping to be expressive enough, the reservoir dimension must be sufficiently high. In effect, RC transforms a low-dimensional nonlinear problem into a high-dimensional linear one.

Moreover, a notable strength of reservoir computing is that the reservoir function $\mathbf{f}$ can be chosen quite freely — it does not even need to be precisely known. This flexibility enables the use of physical systems as reservoirs, particularly in photonic or optical platforms, which offer high bandwidth and fast computation speeds²⁹.

2. Next Generation Reservoir Computing (NGRC)

Next-generation reservoir computing (NGRC) is a simplified alternative to traditional RC. A schematic overview of the NGRC one-step prediction mechanism is shown in Fig. 2. Instead of relying on an explicitly simulated dynamical system as the reservoir, NGRC uses a static, implicit reservoir structure based on nonlinear vector autoregression². The central idea is to take the (normalized) time-delay embedded state vector,

$$\mathbf{u}_H(t) := [\mathbf{u}(t), \mathbf{u}(t - \tau), \mathbf{u}(t - 2\tau), \dots, \mathbf{u}(t - (H - 1)\tau)],$$

and map it nonlinearly into a high-dimensional space,

$$\mathbf{r}(t) = P(\mathbf{u}_H(t)),$$

as illustrated in Fig. 1b. The projected state $P(\mathbf{u}_H(t))$ serves as the reservoir state $\mathbf{r}(t)$, forming the implicit

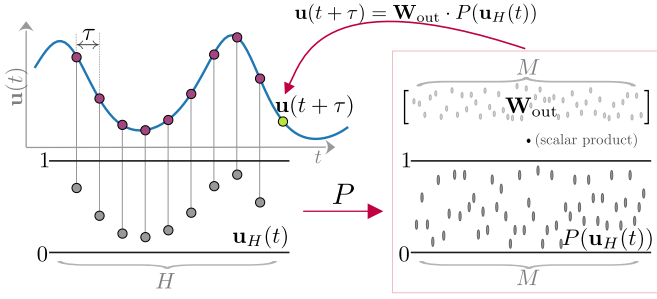


FIG. 2: Schematic of the NGRC one-step predictor. A normalized, time-delay embedded state $\mathbf{u}_H(t)$ is lifted by the nonlinear projection P into a high-dimensional feature vector, which is mapped linearly by $\mathbf{W}_{\text{out}}$ to predict $\mathbf{u}(t + \tau)$. The diagram shows the single scalar signal $\mathbf{u}(t)$ case; for N observables, $\mathbf{u}_H$ has dimension NH and $\mathbf{W}_{\text{out}}$ is an $M \times N$ matrix.

reservoir. Prediction of the next state $\mathbf{u}(t + \tau)$ is then performed — as before in RC — using a simple linear model,

$$\mathbf{u}(t + \tau) = \mathbf{W}_{\text{out}} \cdot P(\mathbf{u}_H(t)), \quad (3)$$

which constitutes the full one-step prediction mechanism of the NGRC approach. The only trainable parameter is the output weight matrix $\mathbf{W}_{\text{out}}$ which can be learned by fitting the linear relation (3) to a training data set.

This approach closely aligns with Koopman operator theory^{30–32}, which asserts that the dynamics of a nonlinear system can be represented by an infinite-dimensional linear operator acting on a suitably chosen set of observables. In NGRC, the delay-embedded state vector $\mathbf{u}_H(t)$ is mapped into a higher-dimensional space, resulting in the state $\mathbf{r}(t) = P(\mathbf{u}_H(t))$, which serves as such a set of observables, effectively lifting the system into a higher-dimensional function space where its evolution becomes approximately linear. Rather than explicitly constructing the Koopman operator, NGRC learns a linear mapping $\mathbf{W}_{\text{out}}$ between $\mathbf{r}(t)$ and the predicted next state $\mathbf{u}(t + \tau)$. The linearity of $\mathbf{W}_{\text{out}}$ is consistent with the Koopman framework, but NGRC sidesteps the need to estimate the full operator. Finally, we note that, while this operation often is referred to as a “projection” in literature, we may more aptly think of this operation as a “lifting”, as it maps the state into a higher-dimensional space rather than projecting onto a subspace.

An advantage of the NGRC architecture is that the time-delay embedding is explicit, eliminating the long transients required in traditional RC to forget initial conditions. This simplification makes NGRC more practical for manipulating and initializing states, particularly in digital twin settings, where constructing meaningful initial conditions is often intractable due to the “black box” nature of the reservoir.

Despite its simplicity, NGRC has demonstrated strong performance on several challenging tasks², including (1)

short-term forecasting, (2) attractor reconstruction, and (3) generalization to unseen regions of the state space.

III. METHODS

A. Scope of This Work

We introduce a simple, scalable projection scheme to construct the high-dimensional feature space used in next-generation reservoir computing. This projection involves **pseudo-random mixing of the delay-embedded measurement vector $\mathbf{u}_H$** , and is detailed in Sec. IIIB.

Using this projection, we train NGRC models on several representative dynamical systems, leveraging both complete and partial observations. We demonstrate that these models accurately reproduce short-term trajectories, long-term attractors, asymptotic phases, and bifurcation diagrams. Notably, we show that bifurcation diagrams can be inferred even when the bifurcation parameter varies in time as a correlated stochastic process — a scenario often arising in non-stationary data collection conditions, similar to the recurrent neural network approach in Ref.³³.

Our NGRC models remain stable over long prediction horizons and do not exhibit any transversal instabilities — it seems that, even asymptotically, the integrator does not drift away and remains on the same attractor. This stability enables us to estimate bifurcation structure directly from a single long trajectory, employing a quasi-continuation approach.

B. Projection P

In Ref.², the projection is achieved by constructing the Volterra series of the measurements $\mathbf{u}_H$, a series which contains monomials formed by multiplying the components of the input signal. In Ref.³⁴, Chebyshev polynomials were used instead. Other NGRC lifting formulations, including kernel-based variants, have also been studied^{35,36}. Various processes may thus be employed for this projection, but **a limitation of such polynomial-based expansions is that the projection dimension grows with the size of the input vector, so increasing the observation dimension or embedding depth directly increases the number of features**. To avoid this dependency, we adopt a **pseudorandom projection scheme** based on pairwise mixing of input components. This approach makes the feature dimension M a tunable parameter independent of the input size, thereby offering scalability across different system dimensions.

Our aim in introducing the pseudorandom projection is not to outperform existing feature constructions such as Chebyshev or Volterra expansions. Polynomial-based schemes are known to have near-optimal approximation properties for smooth functions on bounded intervals, so one should not expect a pseudorandom feature map to

achieve lower approximation error for a given feature dimension M . The motivation for our construction is different: it permits M to be adjusted continuously and independently of the input dimension, without requiring discrete jumps between entire sets of basis functions (e.g. choosing all triplets instead of all pairs sees the number of final features jump drastically from a square number to a cube number). This flexibility is often advantageous in practice, since any loss in approximation efficiency relative to polynomial bases can be offset by modestly increasing M .

The projection P embeds (lifts) the state vector $\mathbf{u}_H$ into a higher-dimensional space, with $P(\mathbf{u}_H) = [p_1, p_2, p_3, \dots, p_M]$, where the dimension $M \gg 1$ is a free parameter, typically chosen to be sufficiently large. For this task, we construct the projection using a pseudo-random shuffled sequence of indices, which is deterministic and reproducible.

First, we normalize the values of the input vector $\mathbf{u}_H$ to lie within the unit interval $[0, 1]$, as illustrated in Fig. 2. To maintain numerical stability, a small positive margin $\epsilon > 0$ is introduced, ensuring the values fall within $(\epsilon, 1 - \epsilon)$ and never leave the unit interval during computation.

The measurement vector $\mathbf{u}$ is N -dimensional, and the time-delay embedded vector $\mathbf{u}_H$ contains H delayed samples, resulting in a complete input vector of size $N \times H$ components:

$$\begin{bmatrix} u_1(t), u_1(t - \tau), \dots, u_1(t - (H - 1) \cdot \tau), \\ u_2(t), u_2(t - \tau), \dots, u_2(t - (H - 1) \cdot \tau), \\ \vdots \\ u_N(t), u_N(t - \tau), \dots, u_N(t - (H - 1) \cdot \tau) \end{bmatrix}. \quad (4)$$

These initial values form the starting point for constructing the projected vector $P(\mathbf{u}_H)$. To distinguish them from the new features generated during the projection process, we assign non-positive indices:

$$p_{-(N \cdot H - 1)}, p_{-(N \cdot H - 2)}, \dots, p_{-1}, p_0,$$

where, for example, $p_{-(N \cdot H - 1)} = u_1(t)$ and $p_0 = u_N(t - (H - 1) \cdot \tau)$.

Next, we generate the projected vector of size M (with $M \gg N \times H$), by iteratively creating new values, i.e., incrementing $m = 1, \dots, M$, from component pairs p_i and p_j with $i, j < m$. Each new component is defined by the nonlinear combination,

$$p_m = (1 - p_i)^{p_j}, \quad (5)$$

where the indices i and j are chosen from a pseudo-random but deterministic sequence, seeded for reproducibility. Any duplicate values are discarded to prevent degeneracy and to ensure that feature matrix remains full-rank during training.

To illustrate this process, the first new value, p_1 , is created by combining two pseudo-randomly selected elements from the initial set of values with non-positive indices $(p_i, p_j$ where $i, j \leq 0$). The next value, p_2 , can now

incorporate p_1 along with the original components, and this process continues. Consequently, the pool of available elements grows with each step, rapidly increasing the diversity of possible pairings used in the projection.

Importantly, the map (5) preserves the unit interval: if $p_i, p_j \in (0, 1)$, then p_m will also remain within that range. This process is repeated until M distinct components are obtained. In practice, we exclude the original components (4) from the final projected vector, although this step is not strictly necessary. The resulting M -dimensional vector $[p_1, p_2, \dots, p_M]$ is then used as the projected state $P(\mathbf{u}_H)$.

A notable property of this projection scheme is that it preserves the unit interval: all projected components lie within $[0, 1]$, are typically of the same order, and do not aggregate around particular values. Figure 3 shows the distribution of p_m values for a trajectory of the Lorenz system. The values are distributed smoothly and densely across the interval, contrasting monomial-based projections which often produce extreme values, either very small (near zero) or excessively large, depending on the scaling of the input signal.

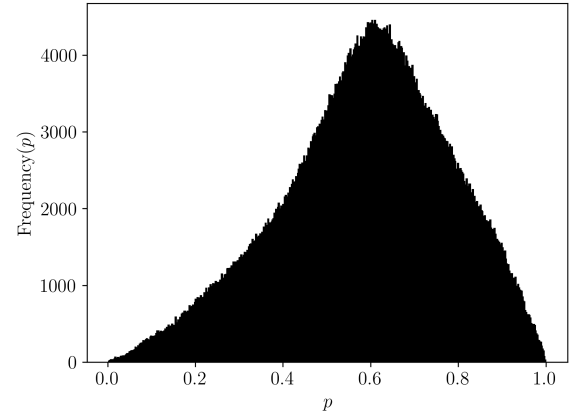


FIG. 3: Distribution of p_m values generated by the projection P for a long time series of the Lorenz system (11). The values remain bounded within the unit interval and are smoothly distributed, reflecting the stable and well-scaled nature of the projection scheme.

C. Optimizing the linear model

Assuming that the projection P has been chosen appropriately, we can expect that the linear relation (3) provides a good approximation of the system dynamics and can be fitted directly to data. Given the observed states $\mathbf{u}(t)$ at several time points, we construct their delay-embedded forms $\mathbf{u}_H(t)$, compute the corresponding projections $P(\mathbf{u}_H(t))$, and then optimize the relation (3) over all available training time points. Fitting the model corresponds to finding the optimal output weight matrix $\mathbf{W}_{\text{out}}$. Since the relation is linear, the opti-

mization is straightforward and can be performed using standard techniques such as least squares regression or singular value decomposition³⁷.

The state vector $\mathbf{u}$ has dimension N , while the projected vector $P(\mathbf{u}_H)$ is a vector of size M , where typically $M \gg 1$ and N may be small (even $N = 1$). Thus, the output matrix $\mathbf{W}_{\text{out}}$ has dimensions $N \times M$. Each time point contributes one instance of the relation (3), and the entire training dataset over T time points can be expressed as:

$$\mathbf{U} = [\mathbf{u}(t + \tau), \mathbf{u}(t + 2\tau), \mathbf{u}(t + 3\tau), \dots, \mathbf{u}(t + T\tau)] \quad (6a)$$

$$\mathbf{P} = [P(\mathbf{u}_H(t)), P(\mathbf{u}_H(t + \tau)), \dots, P(\mathbf{u}_H(t + (T - 1)\tau))] \quad (6b)$$

which yields the matrix relation:

$$\mathbf{U} = \mathbf{W}_{\text{out}} \mathbf{P}. \quad (7)$$

The optimal output weights $\mathbf{W}_{\text{out}}$ are then obtained via least squares minimization:

$$\mathbf{W}_{\text{out}} = \mathbf{U} \mathbf{P}^T (\mathbf{P} \mathbf{P}^T)^{-1}. \quad (8)$$

In practice, it is common to add Tikhonov regularization to improve conditioning and suppress overfitting:

$$\mathbf{W}_{\text{out}} = \mathbf{U} \mathbf{P}^T (\mathbf{P} \mathbf{P}^T + \lambda \mathbf{I})^{-1}, \quad (9)$$

where $\lambda > 0$ is a regularization parameter and $\mathbf{I}$ the identity matrix.

D. Tracking errors

Once the model $\mathbf{W}_{\text{out}}$ has been determined, we evaluate its accuracy by tracking the mean squared error of the one-step prediction. For each time point n , we compare the true next state $\mathbf{u}((n + 1)\tau)$ with the predicted state $\mathbf{W}_{\text{out}} P(\mathbf{u}_H(n\tau))$, where $P(\mathbf{u}_H(n\tau))$ is the projected feature vector constructed from the previous H delay-embedded states. The mean squared error is then

$$E = \frac{1}{T} \sum_{n=1}^T \|\mathbf{u}((n + 1)\tau) - \mathbf{W}_{\text{out}} P(\mathbf{u}_H(n\tau))\|^2, \quad (10)$$

where $\|\cdot\|$ denotes the Euclidean norm and T is the number of prediction steps. This error can be computed both on the training dataset and on a separate validation set that was not used during fitting.

E. Validation

Validation is the assessment of model performance on data not used during training, providing a measure of how well the model generalizes to new, unseen inputs.

The projection dimension M is a free parameter and can be treated as a hyperparameter of the method. While the training error (10) typically decreases with increasing M , monitoring the validation error is essential to avoid overfitting (i.e., as M becomes larger, the model can fit the data, including fluctuations or noise, too closely, effectively “remembering” and thus performs poorly on unseen data in the validation set).

Additionally, the least squares solution relies on the invertibility of the matrix $\mathbf{P} \mathbf{P}^T$, which can become problematic for simple systems or when the projection dimension M is large. A practical way to mitigate this issue is to add a small amount of measurement noise to the original state vectors $\mathbf{u}$, which improves numerical stability and can also act as a form of regularization. Conditioning issues and the sensitivity of NGRC regression to the lifted feature representation have been analyzed in Ref.³⁸, showing how noise and regularization can improve numerical stability.

Figure 4 shows the training and validation errors (10) as a function of M , for two different levels of Gaussian measurement noise. In the noise-free case, the error decreases only up to about $M = 200$, after which it increases sharply, leading to models that are effectively unusable. In contrast, adding 1% Gaussian noise stabilizes the inference and prevents this error blow-up for moderate values of M . However, excessive noise can also degrade model quality, so we selected the 1% level as a reasonable compromise — at which point the training and validation errors remain comparable across a range of M . We consistently use 1% measurement noise for all examples going further.

F. Long-term stability

While one-step prediction accuracy is informative, it does not guarantee reliable long-term behavior when the predictor is iterated autonomously. Transverse instability has been identified as an important limitation for NGRC-type models in long-term autonomous prediction³⁴, and recent work has examined both intrinsic stability conditions³⁹ and complementary post-processing strategies for enhancing long-horizon robustness⁴⁰. Similar stability concerns have also been reported in classical RC, where training noise has been shown to act as a regularizer that improves long-horizon autonomous behavior⁴¹. We therefore assess long-term stability under three regularization settings: (a) Tikhonov regularization with $\lambda = 0.01$ and no measurement noise, (b) the same Tikhonov regularization with 0.1% measurement noise added to the training inputs, and (c) 1% measurement noise without Tikhonov regularization. For each setting we train 100 independent NGRC models and, for each model, generate 10 autonomous prediction trajectories from slightly perturbed initial conditions.

Figure 5 shows representative prediction trajectories together with a reference trajectory from the true ODE

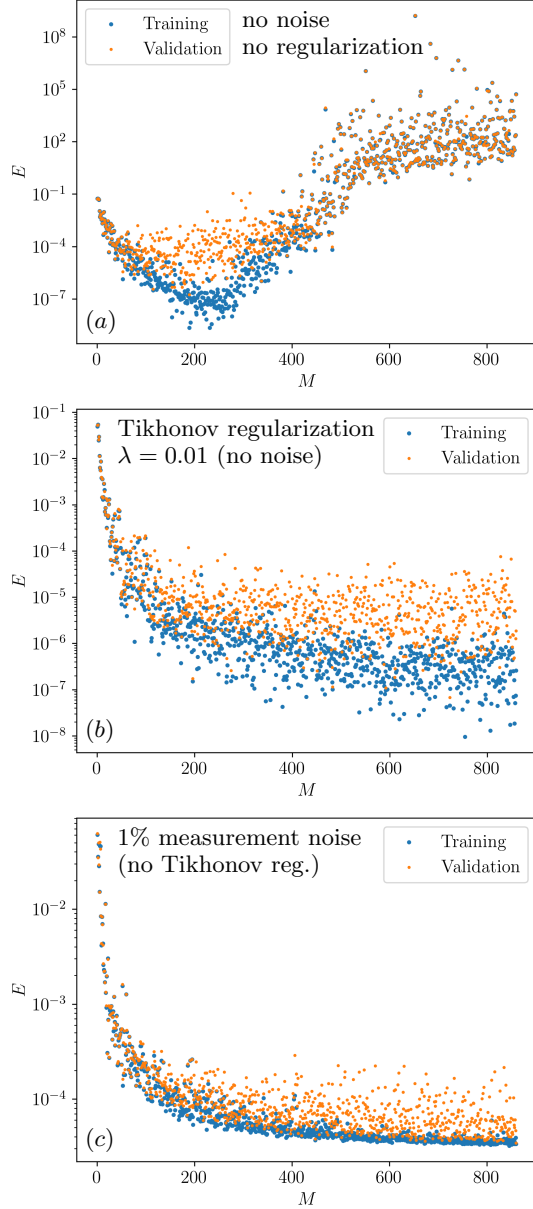


FIG. 4: Mean-squared one-step prediction error as a function of the projection dimension M under three regularization scenarios: (a) no regularization, (b) Tikhonov regularization with $\lambda = 0.01$, and (c) addition of 1% Gaussian measurement noise to the training inputs. (a) Without regularization, errors initially decrease but instabilities appear for larger M . (b) Tikhonov regularization suppresses high- M instabilities, but mild overfitting persists, with training errors consistently below validation errors. (c) Adding measurement noise acts as an implicit regularizer by broadening the sampled state space, yielding training and validation errors that remain comparable across the full range of M . This motivates the use of 1% measurement noise in subsequent experiments as a practical balance between stability and predictive accuracy. Excessive noise, however, degrades performance, so a trade-off applies. All results use long trajectories ($T = 10^5$) for both training and validation.

system (here we used Lorenz (11)). Because all signals are normalized to lie in $[0, 1]$, we consider a trajectory to have diverged once it leaves this interval and terminate its evolution at that point. The fraction of escaped trajectories is plotted alongside the trajectories.

In case (a), most trajectories exhibit rapid transverse instability and leave the admissible range after only a few steps. In case (b), even a very small amount of measurement noise noticeably improves stability, reducing the escape rate and allowing trajectories to remain near the attractor for longer times. In case (c), training with 1% measurement noise yields the most robust behavior: over the time window tested, none of the trajectories escapes the interval, despite the absence of Tikhonov regularization. This suggests that measurement noise acts as an effective implicit regularizer by broadening the sampled state space and suppressing fragile directions. For this reason, we adopt 1% measurement noise in all subsequent experiments.

IV. EXAMPLES

A. Reconstruction of attractor from partial observations

We train the reservoir model (3) using the random projection P described in Sec. III B. The training data comes from two standard chaotic systems, the Lorenz system and the Rössler system, which are three dimensional models formulated in variables $x(t), y(t), z(t)$ see Eqs. (11) and Eqs. (12), respectively. We generate long trajectories from each system, but we only use the scalar variable $x(t)$ for training. The model is trained for one-step prediction. After training, it can be used to predict long-term evolution by iterating its own outputs.

First, we compare short-time predictions from the reservoir model with the true trajectories. Next, we let the model run freely for a long time and plot the time-embedded trajectory alongside one from the true system.

For the experiments, we fix some (hyper-)parameters: $M = 1000$ reservoir nodes, 1% measurement noise, and a timeseries length $T = 10^5$. We observe only one variable (x) from each system, so $N = 1$.

1. Lorenz system

We consider the equations

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}\tag{11}$$

with the standard parameters $\sigma = 10, \rho = 28, \beta = 8/3^{42}$.

Figure 6 shows the true Lorenz trajectory (test data) alongside the trajectory generated by the reservoir model using iterative one-step predictions. As expected for a chaotic system, the two trajectories diverge from each

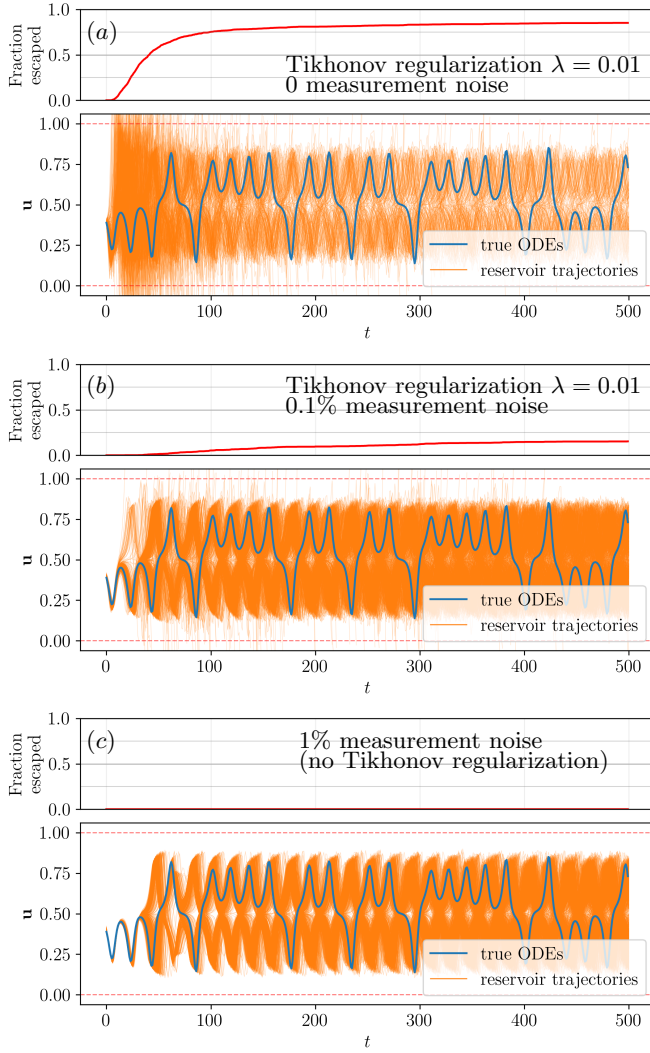


FIG. 5: Long-term autonomous prediction test for NGRC under three regularization settings: (a) Tikhonov regularization with $\lambda = 0.01$ and no measurement noise, (b) the same Tikhonov regularization with 0.1% measurement noise added to the training inputs, and (c) 1% measurement noise without Tikhonov regularization. For each setting, 100 independently trained models are rolled out from 10 slightly perturbed initial conditions, and trajectories are compared with a reference solution of the true ODE system (11). Because all signals are normalized to $[0, 1]$, a trajectory is considered divergent once it leaves this interval, at which point the rollout is terminated. The fraction of runs that diverged is shown alongside the trajectories. (a) sees rapid transverse instability and early divergence in most runs; (b) shows substantially reduced escape rates; (c) demonstrates the strongest stability, with no trajectories leaving escaping over the displayed time window.

other after some time, where small differences amplify exponentially in time. This divergence occurs on a timescale comparable to the Lyapunov time due to the finite accuracy of our one-step predictor.

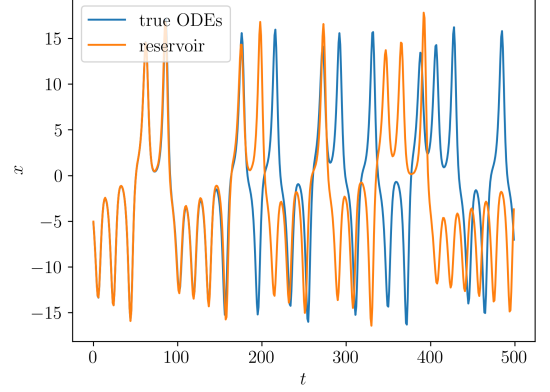


FIG. 6: Short-term trajectories of the Lorenz system. The true trajectory is shown in blue, and the reservoir model prediction in orange. The model was trained using only the scalar variable $x(t)$. Time axis is units of time-sampling step $\tau = 0.04$.

Figure 7 compares the true Lorenz attractor obtained via numerical integration of Eqs. (11) with the one reconstructed from the reservoir model using delay embedding. The reconstruction produces a good match.

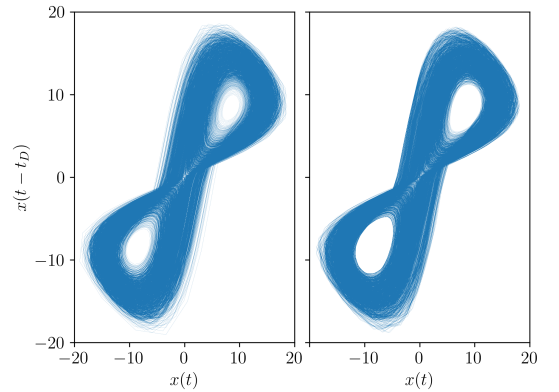


FIG. 7: Delay-embedded Lorenz attractor reconstruction. The true attractor is shown on the left, and the reconstructed attractor on the right. The model was trained using only the scalar variable $x(t)$. Time delay for plot is $3\tau = 0.12$.

2. Rössler system

We consider the equations

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + ay \\ \dot{z} &= b + z(x - c)\end{aligned}\tag{12}$$

with parameters $a = 0.2, b = 0.4, c = 5.7^{43}$.

Figure 8 shows the true Rössler trajectory, obtained by numerically integrating Eqs. (12), and the one produced by the reservoir model through iterative one-step predictions. As with the Lorenz case, the trajectories diverge after some time, consistent with the system's chaotic nature.

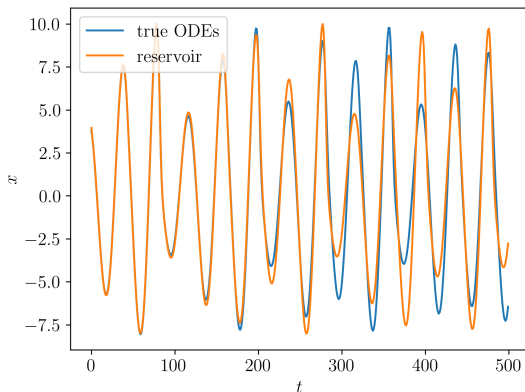


FIG. 8: Short-term trajectories of the Rössler system. The true trajectory is shown in blue, and the reservoir model prediction in orange. The model was trained using only the scalar variable $x(t)$. Time axis is units of time-sampling step $\tau = 0.15$.

Figure 9 compares the true Rössler attractor with the attractor reconstructed from the reservoir model using delay embedding. Again, the original and reconstructed attractors match very well.

B. Reconstruction of bifurcation diagram

We consider the Rössler system (12) and add a variable control parameter η to the right-hand-side of the y equation. The reservoir then observes the resulting system states and learns to reproduce the system's behavior for different η . The variation of η allows us to probe the bifurcation structure of the system. In the following, we first use several data sections corresponding to different constant η values to reconstruct the bifurcation diagram, and secondly, consider a time-dependent, stochastic $\eta(t)$ to mimic less controlled real world situations.

First, we generate trajectories for four different constant values of η (indicated by vertical lines in Fig. 10) and use them to train a single reservoir model. After

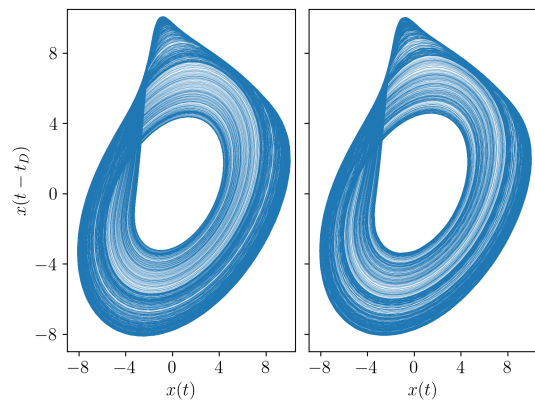


FIG. 9: Delay-embedded Rössler attractor reconstruction. The true attractor is shown on the left, and the reconstructed attractor on the right. The model was trained using only the scalar variable $x(t)$. Time delay for plot is $8\tau = 1.2$.

training, we estimate the bifurcation diagram by varying η in the trained reservoir. The diagram is constructed by plotting the local maxima of the variable $x(t)$ over time, see Fig. 10. The diagram is obtained from only one reservoir trained on data corresponding to four values of η as input to the reservoir.

The reconstruction is accurate near the trained parameter values, and is well interpolated between them. As η moves farther away from η values used for training, the reservoir-generated attractors increasingly deviate from the true system attractor. This is expected, since the model extrapolates beyond the regimes it has seen during training. Such deviations are a common limitation of data-driven models that rely on local interpolation rather than a global parameterization of the underlying dynamics. In particular, NGRC has no built-in mechanism to guarantee correct dynamics outside the region explored during training, and uncertainties can grow rapidly once the model leaves this domain. In Fig. 10, this is most prominent near $\eta = -1$, where the reservoir predicts qualitatively different behavior⁴⁴.

Next, we consider a more challenging and practically relevant scenario. In many real-world applications, it is not possible to collect data under strictly controlled conditions where the control parameter, η , remains constant. Instead, the parameter often varies in time and can be measured only as a noisy signal.

To mimic this situation, we drive the Rössler system (12) with a stochastic parameter $\eta(t)$ evolving according to an Ornstein-Uhlenbeck (OU) process⁴⁵. The OU process is a standard way to model noise with a finite correlation time. It describes a fluctuating variable that is randomly driven but does not change abruptly, producing smoothly varying noise rather than instantaneous

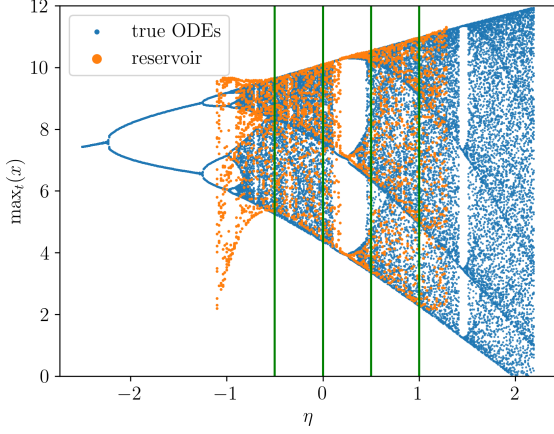


FIG. 10: Bifurcation diagram of the true Rössler system (12) (blue) and the reconstruction from the reservoir (orange). Green vertical lines mark the four input parameter values η used for training. The diagram is computed by plotting the local maxima of $x(t)$. Training used long trajectories ($T = 10^5$), all three state variables (x, y, z), reservoir size $M = 6000$, delay dimension $H = 25$, and 1% measurement noise.

(white-noise) jumps.

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + ay + \eta \\ \dot{z} &= b + z(x - c) \\ \dot{\eta} &= \frac{1}{\tau_c} (-\eta + \rho\xi)\end{aligned}\quad (13)$$

where $\xi(t)$ is Gaussian white noise, and the parameters τ_c and ρ control the correlation time and amplitude of the OU process, respectively. The resulting system has a time-varying, stochastic control parameter, making the learning task harder. We use the trajectories generated by this noise-driven system as training data for a single reservoir model.

In this case as well, the reservoir accurately reproduces the bifurcation structure near trained values of η , demonstrating that even from trajectories with rapidly time-varying control parameters, it can learn the correct asymptotic dynamics corresponding to fixed parameter values. Deviations arise primarily when extrapolating beyond the training range, see Fig. 11. This again reflects the fact that the model is asked to infer behavior for parameter values that are only sparsely or indirectly represented in the training data.

C. Estimation of asymptotic phases

We again consider the noisy Rössler system (13), however, now in a parameter regime where the deterministic system exhibits a stable period-1 limit cycle: $a = 0.2$, $b = 1.6$, $c = 5.7$. We train a reservoir model on time

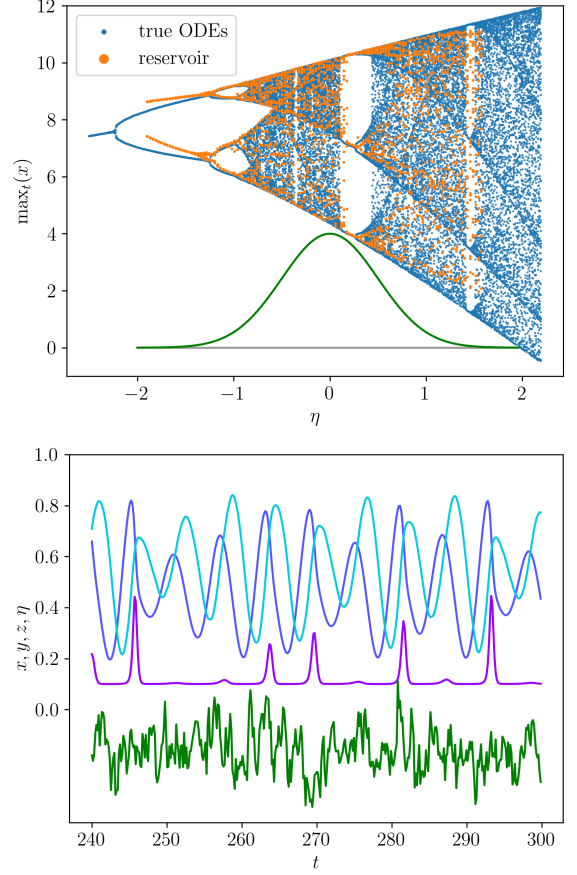


FIG. 11: Top panel: bifurcation diagram of the true Rössler system (12) (blue) and the reconstruction from the reservoir (orange). The training data were obtained from a Rössler system perturbed by an Ornstein-Uhlenbeck input (Eq. (13)). The green curve shows the empirical probability density of the noise variable η . The bifurcation points are computed by plotting the local maxima of $x(t)$. Training used a long trajectory ($T = 10^5$), all three variables (x, y, z), reservoir size $M = 6000$, delay dimension $H = 25$, and 1% measurement noise. Bottom panel: the training signal, Rössler variables (x, y, z) are shown in blue, cyan, and purple, and the OU input $\eta(t)$ in green (vertically shifted for clarity).

series of the variables $x(t)$, $y(t)$, and $z(t)$. Unlike in the bifurcation diagram experiment (Sec. IV B), we do not provide the noise signal $\eta(t)$ as an explicit input to the reservoir. Instead, $\eta(t)$ only perturbs the training trajectory (and is thus hidden from the reservoir input), introducing variability that helps the data explore the state space more broadly. The reservoir is expected to “average out” this noise and approximate the dynamics of the unperturbed Rössler system over a wider range of states.

We then evaluate the model’s ability to recover asymptotic phase. This is a hard task to accomplish, as it

requires coherent long-term prediction and a globally accurate model of the evolution of the system. To test this, we generate a new trajectory from the noisy Rössler system (13) — one that has not been used in the training phase — to use as test data. For each point along this trajectory, we evolve the system forward using the reservoir model until it converges to the limit cycle, and then estimate its asymptotic phase from the position on the cycle. The same is done with the true ODEs for comparison. We visualize the estimated phases along the test trajectory using a color palette. The reservoir accurately recovers the phase structure, except in regions of state space that were underrepresented in the training data; see Fig. 12.

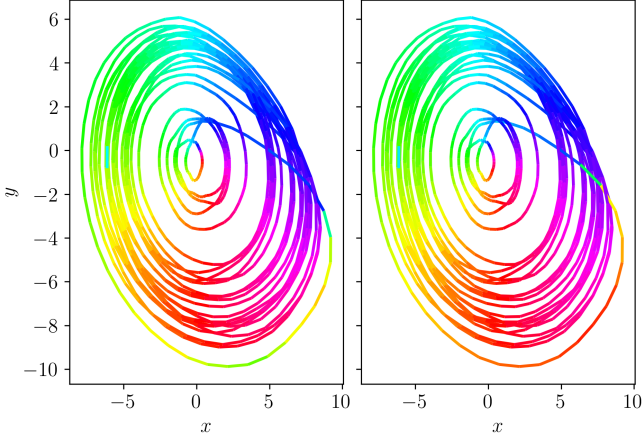


FIG. 12: Asymptotic phase estimated along a test trajectory not used during training. Left: true phase obtained by evolving each state forward using the Rössler ODE system (12) until convergence to the limit cycle. Right: phase estimated by evolving the same states with the trained reservoir model. Color indicates the recovered phase. The reservoir accurately captures the phase structure, except in regions that were poorly sampled during training (see, for example, the rare trajectory excursion in the lower region of the state space).

V. DISCUSSION

In our work, we built on the next-generation reservoir computing framework (NGRC) and proposed a projection method based on pseudorandom nonlinear features. An advantage of this approach is that the number of projected features M can be set independently of the input dimension, whereas in polynomial-based projections the feature space typically depends on the input size $N \cdot H$. We demonstrated the utility of this scalable scheme across several canonical examples, including attractor reconstruction, bifurcation diagram estimation, and asymptotic phase inference. The models showed ro-

bust performance even when trained on limited or non-stationary data, and they could generalize to parameters and dynamics not seen during training. These features suggest that next-generation reservoir computing may be a practical tool for applications ranging from digital twins and surrogate modeling to problems in climate science, finance, and biology. Reservoir computing has also been applied to real-world forecasting problems, such as anticipating food price fluctuations in agricultural markets⁴⁶.

One of the central advantages of NGRC over classical reservoir computing lies in its transparency and manipulability. Because NGRC evolves entirely in the space of observable inputs, it allows direct and global perturbations of the system state within this observable space (i.e., by modifying the delay-embedded measurement vector $\mathbf{u}_H$). This feature is particularly valuable for digital twin and surrogate modeling tasks, where probing system responses to hypothetical inputs or counterfactual states (i.e., hypothetical system states not visited in the observed trajectory) is essential. In contrast, classical RC relies on evolving a high-dimensional reservoir state $\mathbf{r}(t)$, which is difficult to interpret or initialize, making it challenging to explore off-attractor or perform controlled interventions.

Another strength of NGRC is its use of explicit time-delay embedding, which eliminates the need for long transients to “wash out” initial conditions, as required in classical RC. This approach makes it easier to analyze the model, set initial states, and perform controlled simulations. Even though the projection $P(\mathbf{u}_H)$ may involve a black-box transformation, the evolution remains anchored in physically meaningful variables, enhancing interpretability and stability.

A known challenge in NGRC, highlighted in Ref.³⁴, is transversal instability in long-term autonomous prediction, whereby small perturbations orthogonal to the attractor can grow and lead to rapid divergence. We examined this issue directly and found that, although Tikhonov regularization helps mitigate overfitting and numerical ill-conditioning in one-step regression — consistent with conditioning effects reported in Ref.³⁸ — it does not prevent long-term instabilities: models trained without noise generally diverge quickly when iterated autonomously, even when regularized. By contrast, adding a small amount of measurement noise to the training inputs reliably suppresses such failures (similar stabilizing effects were reported in classical RC⁴¹). Post-processing approaches for enhancing long-horizon robustness have also been considered⁴⁰. Across all systems tested, we did not observe any transverse instability once 1% measurement noise was included, even for extremely long autonomous rollouts used in our quasi-continuation bifurcation reconstructions. A plausible explanation is that noise spreads the training samples slightly off the attractor surface, so the regression must learn a mapping that remains locally consistent not only along the attractor but also in nearby transverse directions (see also the structural stability considerations discussed in Ref.³⁹).

Deterministic trajectories sample only a thin manifold and leave transverse directions weakly constrained; noise broadens this sampling, which empirically yields models that do not amplify small off-manifold deviations. We therefore use 1% input noise as our default configuration in all experiments.

One promising direction for future work is to apply NGRC to network inference. By training NGRC models on multivariate time series from interacting systems, one can construct a complete model of the network, which can then be interrogated using surrogate modeling or digital twin techniques to estimate both the intrinsic dynamics of individual units as well as their functional connectivity patterns.

DATA AVAILABILITY STATEMENT

Data sharing is not applicable to this article as no new data were created or analyzed in this study. For reproducibility, all code used in this work is publicly available at <https://github.com/rok-cestnik/next-gen-res-comp>, Ref.⁴⁷.

ACKNOWLEDGMENTS

We thankfully acknowledge useful discussions with Irmantas Ratas, Andrew Flynn, Edmilson Roque dos Santos and Herbert Jaeger. We acknowledge financial support from the Crafoord Foundation (project no. 20240689).

- ¹Z. Lu, B. R. Hunt, and E. Ott, “Attractor reconstruction by machine learning,” *Chaos* **28** (2018), 10.1063/1.5039508.
- ²D. J. Gauthier, E. Bollt, A. Griffith, and W. A. Barbosa, “Next generation reservoir computing,” *Nature Communications* **12** (2021), 10.1038/s41467-021-25801-2.
- ³H. Jaeger, “The “Echo State” Approach to Analysing and Training Recurrent Neural Networks,” Technical Report GMD Report 148 (GMD — German National Research Center for Information Technology, 2001).
- ⁴W. Maass, T. Natschl ger, and H. Markram, “Real-time computing without stable states: A new framework for neural computation based on perturbations,” *Neural Computation* **14**, 2531–2560 (2002).
- ⁵M. Luk  evi  ius and H. Jaeger, “Reservoir computing approaches to recurrent neural network training,” *Computer Science Review* **3**, 127–149 (2009).
- ⁶G. Tanaka, T. Yamane, J. B. H  roux, R. Nakane, N. Kanazawa, S. Takeda, H. Numata, D. Nakano, and A. Hirose, “Recent advances in physical reservoir computing: A review,” *Neural Networks* **115**, 100–123 (2019).
- ⁷K. Nakajima and I. Fischer, *Reservoir computing* (Springer, 2021).
- ⁸L. Appeltant, M. C. Soriano, G. Van der Sande, J. Danckaert, S. Massar, J. Dambre, B. Schrauwen, C. R. Mirasso, and I. Fischer, “Information processing using a single dynamical node as complex system,” *Nature Communications* **2**, 468 (2011).
- ⁹N. D. Haynes, M. C. Soriano, D. P. Rosin, I. Fischer, and D. J. Gauthier, “Reservoir computing with a single time-delay autonomous boolean node,” *Phys. Rev. E* **91**, 020801 (2015).
- ¹⁰L. Larger, A. Bayl  n-Fuentes, R. Martinenghi, V. S. Udaltsov, Y. K. Chembo, and M. Jacquot, “High-speed photonic reservoir computing using a time-delay-based architecture: Million words per second classification,” *Phys. Rev. X* **7**, 011015 (2017).
- ¹¹C. Fernando and S. Sojakka, “Pattern recognition in a bucket,” in *European Conference on Artificial Life* (Springer, 2003) pp. 588–597.
- ¹²L. Domingo, M. Grande, G. Carlo, F. Borondo, and J. Borondo, “Optimal quantum reservoir computing for market forecasting: An application to fight food price crises,” (2023), arXiv:2401.03347 [quant-ph].
- ¹³H. Jaeger and H. Haas, “Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication,” *Science* **304**, 78–80 (2004).
- ¹⁴F. Wyffels and B. Schrauwen, “A comparative study of reservoir computing strategies for monthly time series prediction,” *Neurocomputing* **73**, 1958–1964 (2010).
- ¹⁵J. Pathak, Z. Lu, B. R. Hunt, M. Girvan, and E. Ott, “Using machine learning to replicate chaotic attractors and calculate lyapunov exponents from data,” *Chaos* **27**, 121102 (2017).
- ¹⁶Z. Lu, J. Pathak, B. Hunt, M. Girvan, R. Brockett, and E. Ott, “Reservoir observers: Model-free inference of unmeasured variables in chaotic systems,” *Chaos* **27**, 041102 (2017).
- ¹⁷R. S. Zimmermann and U. Parlitz, “Observing spatio-temporal dynamics of excitable media using reservoir computing,” *Chaos* **28**, 043118 (2018).
- ¹⁸R. M. Smeal, G. B. Ermentrout, and J. A. White, “Phase-response curves and synchronized neural networks,” *Philosophical Transactions of the Royal Society B: Biological Sciences* **365**, 2407–2422 (2010).
- ¹⁹K. C. Wedgwood, K. K. Lin, R. Thul, and S. Coombes, “Phase-amplitude descriptions of neural oscillator models,” *The Journal of Mathematical Neuroscience* **3**, 1–22 (2013).
- ²⁰B. Pietras and A. Daffertshofer, “Reduced phase models of oscillatory neural networks,” in *Physics of Biological Oscillators: New Insights into Non-Equilibrium and Non-Autonomous Systems* (Springer, 2021) pp. 29–54.
- ²¹R. Cestnik and M. Rosenblum, “Inferring the phase response curve from observation of a continuously perturbed oscillator,” *Sci. Rep.* **8** (2018), 10.1038/s41598-018-32069-y.
- ²²R. Cestnik, E. Mau, and M. Rosenblum, “Inferring oscillator’s phase and amplitude response from a scalar signal exploiting test stimulation,” *New Journal of Physics* **24**, 123012 (2022).
- ²³M. Hermans and B. Schrauwen, “Memory in linear recurrent neural networks in continuous time,” *Neural Networks* **23**, 341–355 (2010).
- ²⁴M. Hermans and B. Schrauwen, “Recurrent kernel machines: Computing with infinite echo state networks,” *Neural Computation* **24**, 104–133 (2012).
- ²⁵L. Grigoryeva and J.-P. Ortega, “Echo state networks are universal,” *Neural Networks* **108**, 495–508 (2018).
- ²⁶L. Grigoryeva, A. Hart, and J.-P. Ortega, “Chaos on compact manifolds: Differentiable synchronizations beyond the takens theorem,” *Phys. Rev. E* **103**, 062204 (2021).
- ²⁷G. Manjunath and H. Jaeger, “Echo state property linked to an input: Exploring a fundamental characteristic of recurrent neural networks,” *Neural Computation* **25**, 671–696 (2013).
- ²⁸I. B. Yildiz, H. Jaeger, and S. J. Kiebel, “Re-visiting the echo state property,” *Neural Networks* **35**, 1–9 (2012).
- ²⁹L. Larger, M. C. Soriano, D. Brunner, L. Appeltant, J. M. Guti  rrez, L. Pesquera, C. R. Mirasso, and I. Fischer, “Photonic information processing beyond turing: an optoelectronic implementation of reservoir computing,” *Optics express* **20**, 3241–3249 (2012).
- ³⁰B. Koopman, “Hamiltonian systems and transformation in hilbert space,” *Proceedings of the National Academy of Sciences* **17**, 315–318 (1931).
- ³¹I. Mezi  , “Spectral properties of dynamical systems, model reduction and decompositions,” *Nonlinear Dynamics* **41**, 309–325 (2005).
- ³²S. L. Brunton, M. Budi  i  , E. Kaiser, and J. N. Kutz, “Modern koopman theory for dynamical systems,” *SIAM Review* **64**, 229–

- 340 (2022).
- ³³R. Cestnik and M. Abel, “Inferring the dynamics of oscillatory systems using recurrent neural networks,” *Chaos* **29**, 063128 (2019).
 - ³⁴I. Ratas and K. Pyragas, “Application of next-generation reservoir computing for predicting chaotic systems from partial observations,” *Phys. Rev. E* **109**, 064215 (2024).
 - ³⁵L. Grigoryeva, H. L. J. Ting, and J.-P. Ortega, “Infinite-dimensional next-generation reservoir computing,” *Phys. Rev. E* **111**, 035305 (2025).
 - ³⁶L. Gonon, L. Grigoryeva, and J.-P. Ortega, “Reservoir kernels and volterra series,” (2025), arXiv:2212.14641 [cs.LG].
 - ³⁷W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes 3rd Edition: The Art of Scientific Computing*, 3rd ed. (Cambridge University Press, USA, 2007).
 - ³⁸Y. Zhang, E. Roque dos Santos, H. Zhang, and S. P. Cornelius, “How more data can hurt: Instability and regularization in next-generation reservoir computing,” *Chaos* **35**, 073102 (2025).
 - ³⁹E. R. dos Santos and E. Boltt, “On the emergence of numerical instabilities in next generation reservoir computing,” (2025), arXiv:2505.00846 [stat.ML].
 - ⁴⁰T. Berry and S. Das, “Limits of learning dynamical systems,” *SIAM Review* **67**, 107–137 (2025).
 - ⁴¹A. Wikner, J. Harvey, M. Girvan, B. R. Hunt, A. Pomerance, T. Antonsen, and E. Ott, “Stabilizing machine learning prediction of dynamics: Novel noise-inspired regularization tested with reservoir computing,” *Neural Networks* **170**, 94–110 (2024).
 - ⁴²E. N. Lorenz, “Deterministic nonperiodic flow,” *Journal of the Atmospheric Sciences* **20**, 130–141 (1963).
 - ⁴³O. E. Rössler, “An equation for continuous chaos,” *Physics Letters A* **57**, 397–398 (1976).
 - ⁴⁴J. O’Hagan, A. Keane, and A. Flynn, “Confabulation dynamics in a reservoir computer: Filling in the gaps with untrained attractors,” *Chaos* **35**, 093130 (2025).
 - ⁴⁵G. E. Uhlenbeck and L. S. Ornstein, “On the theory of the brownian motion,” *Phys. Rev.* **36**, 823–841 (1930).
 - ⁴⁶L. Domingo, M. Grande, F. Borondo, and J. Borondo, “Anticipating food price crises by reservoir computing,” *Chaos, Solitons & Fractals* **174**, 113854 (2023).
 - ⁴⁷R. Cestnik, “github.com/rok-cestnik/next-gen-res-comp,” GitHub repository (2025).